HuggingFace 실습

학습 목표 및 학습 내용

- Hugging Face를 사용하여 감정 분석을 수행해 봅니다.

학습 내용

- Hugging Face 라이브러리 설치
- Hugging Face API 토큰 인증
- 모델 로드 및 감정 분석 수행
- 다른 언어(특히 한국어)에 대한 감정 분석

개발 환경

- 실습 전체는 CPU 환경에서도 무리 없이 가능
- 특히 한국어 모델은 크기가 큰 경우가 많아 GPU 사용 시 훨씬 빠름.

- 라이브러리 설치

```
!pip install transformers torch
!pip install huggingface_hub
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.35.3)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transfor
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.10.5)
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.12/dist-packages (0.35.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (3.20.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (2025.3.0)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (2.32.4)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (1.1.10)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_hub) (2025.10.5)
```

```
import huggingface_hub
import transformers
print(huggingface_hub.__version__)
print(transformers.__version__)
```

```
0.35.3
4.57.0
```

Hugging Face 토큰 받기 및 인증

- 01 https://huggingface.co/ 에 접속하여 Hugging Face 계정 생성
- 02 계정이 생성되면 우측 상단의 프로필 아이콘을 클릭합니다.
- 03 메뉴에서 "Access tokens" 탭으로 이동합니다.
- "Access tokens" 버튼을 클릭하여 API 토큰을 생성합니다.
- "Create token" 버튼을 클릭하여 토큰을 생성합니다.
    - "Create token" 버튼이 활성화가 안되었을 경우, 이메일 인증이 완료되었는지 꼭 체크하고 확인합니다.
- 토큰은 한번만 복사가 가능하기에 임의의 장소에 복사해 둡니다.

```
from huggingface_hub import login
from transformers import pipeline

# Hugging Face 토큰 입력 받기
from getpass import getpass

huggingface_token = getpass("Hugging Face 토큰을 입력하세요: ")

# Hugging Face 로그인
login(token=huggingface_token)
```

```
Hugging Face 토큰을 입력하세요: · · · · · · · · · · 
```

- 정상적으로 로그인된 계정 정보가 출력되면 로그인 성공입니다!

```
from huggingface_hub import whoami

info = whoami()
print(info['type'])
```

```
user
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

모델 로드 및 감정 분석 수행하기

- DistilBERT는 BERT 모델(구글 2018년)을 기반으로 만들어진 경량화된 버전
- 감정 분석 모델을 사용하여 주어진 문장 "I love using Hugging Face Transformers!"에 대한 감정 분석을 수행

```
# 모델 로드
model_name = "distilbert-base-uncased"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트 - 감정분석
print(classifier("I love using Hugging Face Transformers!"))
```

```
config.json: 100%                                                 483/483 [00:00<00:00, 51.5kB/s]
model.safetensors: 100%                                           268M/268M [00:08<00:00, 20.6MB/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
tokenizer_config.json: 100%                                       48.0/48.0 [00:00<00:00, 3.25kB/s]
vocab.txt: 100%                                                   232k/232k [00:00<00:00, 2.98MB/s]
tokenizer.json: 100%                                              466k/466k [00:00<00:00, 3.03MB/s]
Device set to use cuda:0
[{'label': 'LABEL_1', 'score': 0.5034177303314209}]
```

결과 이해

- https://huggingface.co/distilbert-base-uncased
- LABEL 1의 확률 _ 해당 라벨일 확률 (약 50.3%)

다른 모델 사용해 보기

- DeBERTa (Decoding-enhanced BERT with disentangled attention)
- 모델 이름: microsoft/deberta-base
    - DeBERTa는 BERT 및 RoBERTa의 개선된 버전입니다.

```
# 모델 로드
model_name = "microsoft/deberta-base"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트
print(classifier("I love using Hugging Face Transformers!"))
```

```
Some weights of DebertaForSequenceClassification were not initialized from the model checkpoint at microsoft/deberta-base and are newly ini
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Device set to use cuda:0
[{'label': 'LABEL_0', 'score': 0.5090393424034119}]
```

감정 분석을 위한 좀 더 개선된 다른 모델

- bert-base-uncased : BERT 모델의 기본 버전
- roberta-base : BERT의 변형으로 성능이 향상된 버전
- distilbert-base-uncased-finetuned-sst-2-english : 감정 분석을 위해 SST-2 데이터셋으로 미세 조정된 DistilBERT 모델
    - SST-2 데이터셋(SST-2, Stanford Sentiment Treebank-2)은 자연어 처리(NLP) 분야에서 감정 분석(Sentiment Analysis)을 수행하는 데 자주 사용되는 데이터셋. 이 데이터셋은 영화 리뷰 텍스트와 그에 따른 감정 레이블을 포함하고 있으며, Stanford Sentiment Treebank의 일부이다.

```
# 모델 로드
model_name = "distilbert-base-uncased-finetuned-sst-2-english"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트 - 감정분석
print(classifier("I love using Hugging Face Transformers!"))
```

```
config.json: 100%                                   629/629 [00:00<00:00, 68.1kB/s]
model.safetensors: 100%                             268M/268M [00:02<00:00, 107MB/s]
tokenizer_config.json: 100%                         48.0/48.0 [00:00<00:00, 4.61kB/s]
vocab.txt: 100%                                     232k/232k [00:00<00:00, 10.0MB/s]
Device set to use cuda:0
[{'label': 'POSITIVE', 'score': 0.9971315860748291}]
```

- 한글은 어떨까?

```
# 모델 로드
model_name = "distilbert-base-uncased-finetuned-sst-2-english"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트 - 감정분석
print(classifier("나는 허깅페이스를 좋아합니다."))
```

```
Device set to use cuda:0
[{'label': 'POSITIVE', 'score': 0.7349096536636353}]
```

```
# 모델 테스트 - 감정분석
print(classifier("오늘은 기분이 우울하다."))
print(classifier("오늘은 기분이 나쁘다."))
print(classifier("비가 오기에 기분이 상쾌하다"))
print(classifier("오늘은 경기에 이겼다."))
```

```
[{'label': 'POSITIVE', 'score': 0.8701741099357605}]
[{'label': 'POSITIVE', 'score': 0.8941265940666199}]
[{'label': 'POSITIVE', 'score': 0.8284294009208679}]
[{'label': 'POSITIVE', 'score': 0.9058271646499634}]
```

- 한글 성능은 조금 부족하다.

한글 성능이 좋은 모델을 살펴보기

- WhitePeak/bert-base-cased-Korean-sentiment : BERT 기반의 한국어 감정 분석 모델
- nlp04/korean_sentiment_analysis_kcelectra : ELECTRA 기반의 한국어 감정 분석 모델 (https://huggingface.co/nlp04/korean_sentiment_analysis_kcelectra)
- dudcjs2779/sentiment-analysis-with-klue-bert-base : KLUE BERT 기반의 한국어 감정 분석 모델 (https://huggingface.co/dudcjs2779/sentiment-analysis-with-klue-bert-base)
- hun3359/klue-bert-base-sentiment - KLUE-BERT 기반, 60가지 세분화 감정 분류 모델
    - 감정 레이블을 매우 세분화해서, 단순 긍정/부정 넘어서 깊이 있는 감정 분석 가능.(https://huggingface.co/hun3359/klue-bert-base-sentiment)
- alsgyu/sentiment-analysis-fine-tuned-model - beomi/KcBERT 기반으로 AI 허브 한국어 감정 데이터셋으로 파인튜닝 (https://huggingface.co/alsgyu/sentiment-analysis-fine-tuned-model)

```
# 모델 로드 - LABEL_0 :negative, LABEL_1 : positive
model_name = "WhitePeak/bert-base-cased-Korean-sentiment"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트 - 감정분석
print(classifier("나는 허깅페이스를 좋아합니다."))
print(classifier("오늘은 기분이 우울하다."))
print(classifier("오늘은 기분이 나쁘다."))
print(classifier("비가 오기에 기분이 상쾌하다"))
print(classifier("오늘은 경기에 이겼다."))
```

```
Device set to use cuda:0
[{'label': 'LABEL_1', 'score': 0.9368826150894165}]
[{'label': 'LABEL_1', 'score': 0.9425995349884033}]
[{'label': 'LABEL_0', 'score': 0.9227980375289917}]
[{'label': 'LABEL_1', 'score': 0.9647601842880249}]
[{'label': 'LABEL_1', 'score': 0.6626952886581421}]
```

```
# 모델 로드
model_name = "dudcjs2779/sentiment-analysis-with-klue-bert-base"
classifier = pipeline("sentiment-analysis", model=model_name)

# 모델 테스트 - 감정분석
print(classifier("나는 허깅페이스를 좋아합니다."))
# 모델 테스트 - 감정분석
print(classifier("오늘은 기분이 우울하다."))
print(classifier("오늘은 기분이 나쁘다."))
print(classifier("비가 오기에 기분이 상쾌하다"))
print(classifier("오늘은 경기에 이겼다."))
```

```
Device set to use cuda:0
[{'label': 'LABEL_1', 'score': 0.5347328782081604}]
[{'label': 'LABEL_2', 'score': 0.9280436635017395}]
[{'label': 'LABEL_2', 'score': 0.925348699092865}]
[{'label': 'LABEL_1', 'score': 0.9042669534683228}]
[{'label': 'LABEL_1', 'score': 0.6695975661277771}]
```

[실습]

- WhitePeak/bert-base-cased-Korean-sentiment의 모델을 활용하여 감정분석 실습을 해 보자.
- nlp04/korean_sentiment_analysis_kcelectra의 모델을 활용하여 감정분석 실습을 해 보자.
- hun3359/klue-bert-base-sentiment의 모델을 활용하여 감정분석 실습을 해 보자.

REF

- distilbert-base-uncased-finetuned-sst-2-english 모델 페이지(https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english)
- WhitePeak/bert-base-cased-Korean-sentiment 모델페이지(https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english)
- nlp04/korean_sentiment_analysis_kcelectra 모델 페이지(https://huggingface.co/nlp04/korean_sentiment_analysis_kcelectra)
- mrm8488/deberta-v3-ft-financial-news-sentiment-analysis 모델 페이지(https://huggingface.co/mrm8488/deberta-v3-ft-financial-news-sentiment-analysis)

결론

이번 노트북에서는 Hugging Face의 다양한 사전 학습된 모델을 활용하여 감정 분석을 수행하는 방법을 학습했습니다. 주요 학습 성과는 다음과 같습니다:

1. **Hugging Face의 도구 사용법 이해**: Hugging Face의 `Transformers` 라이브러리와 `pipeline`을 사용하여 간단하게 감정 분석을 수행하는 방법을 익혔습니다.
2. **다양한 모델 비교**: 여러 NLP 모델(`DistilBERT`, `DeBERTa`, `BERT`, `RoBERTa` 등)을 사용하여 감정 분석을 수행하고, 모델별 성능을 비교해 보았습니다.
3. **언어별 모델 성능 차이 이해**: 영어와 한국어 텍스트에 대해 감정 분석 모델의 성능을 비교하고, 각 언어에 적합한 모델을 선택하는 것이 중요함을 배웠습니다.

추가적으로 탐구해볼 만한 주제

- **다양한 NLP 작업 적용**: 감정 분석 외에도 번역, 요약, 텍스트 생성 등 Hugging Face의 다양한 모델을 활용한 다른 NLP 작업에 대해 탐구.
- **모델 튜닝 및 커스터마이징**: 사전 학습된 모델을 특정 도메인에 맞게 튜닝하고 커스터마이징하는 방법을 연구.
- **다중 언어 지원**: 한국어를 포함한 다양한 언어에 대해 최적의 모델을 탐색하고 적용.

모두 수고하셨습니다. 열심히 해 주셔서 감사합니다.

```
코딩을 시작하거나 AI로 코드를 생성하세요.
```